WorkflowSelectorDialog - ワークフロー選択ダイアログ

1. 概要

1.1 コンポーネントの目的

WorkflowSelectorDialogは、S3に保存された他のMAIML XMLファイルを選択し、現在のPetri Netにワークフローとして追加するためのモーダルダイアログです。フォルダツリーとファイル一覧の2ペイン構成で、ファイルの検索・選択を容易にします。

主要機能:

- フォルダツリー表示（左ペイン）
- ファイル一覧表示（右ペイン）
- ファイル検索
- フォルダ選択によるフィルタリング
- ファイル情報表示（名前、パス、サイズ）
- ワークフロー追加実行

1.2 使用場面

- EditorToolbarの「ワークフロー」ボタンをクリック後、キャンバスをクリックした時
- 他のMAIML XMLファイルを現在のPetri Netに統合したい時

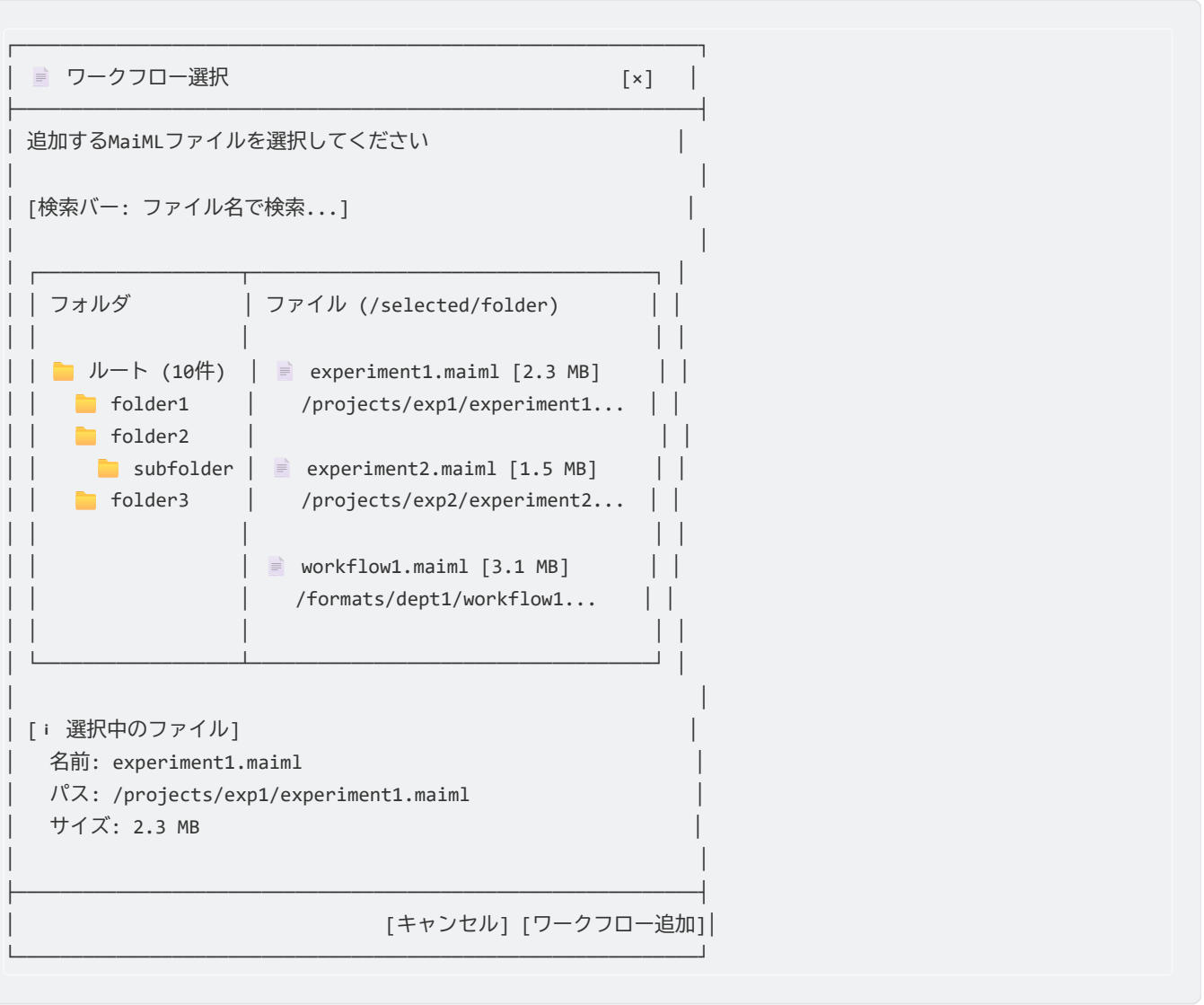
1.3 アクセス権限

- **必須ロール**: Experiment Admin
- **認証**: JWT + Session Hybrid認証

---

2. 画面構成

2.1 レイアウト構造



2.2 主要UI要素

2.2.1 モーダルヘッダー

- **タイトル**: "📄 ワークフロー選択"
- **アイコン**: FileOutlined
- **閉じるボタン**: 右上の×ボタン

2.2.2 説明テキスト

- **テキスト**: "追加するMaiMLファイルを選択してください"
- **タイプ**: secondary

2.2.3 検索バー

- **プレースホルダー**: "ファイル名で検索..."
- **prefix**: SearchOutlinedアイコン
- **allowClear**: true
- **機能**: ファイル名またはパスでフィルタリング

2.2.4 フォルダツリー（左ペイン）

- **幅**: 40%
- **ラベル**: "フォルダ"
- **要素**:
  - Tree（Ant Design）
  - showLine: true
  - showIcon: true
  - アイコン: FolderOutlined（縮小時）、FolderOpenOutlined（展開時）
  - ファイル数Tag: 各フォルダにファイル数を表示

2.2.5 ファイル一覧（右ペイン）

- **幅**: 60%
- **ラベル**: "ファイル (フォルダパス)"
- **要素**:
  - List（Ant Design）
  - bordered: true
  - size: small
- **各ファイル項目**:
  - アイコン: FileOutlined
  - タイトル: ファイル名 + サイズTag
  - 説明: ファイルパス（ellipsis）
  - 選択状態: 青色背景 + 青色ボーダー

2.2.6 空状態表示

- **表示条件**: filteredFiles.length === 0
- **要素**:
  - FolderOutlinedアイコン（64px、灰色）
  - テキスト:
    - 検索またはフォルダ選択時: "ファイルが見つかりません"
    - 初期状態: "フォルダを選択してください"

2.2.7 選択ファイル情報Alert

- **表示条件**: selectedFile !== null
- **タイプ**: info
- **showIcon**: true
- **内容**:
  - 名前
  - パス（ellipsis）
  - サイズ

2.2.8 エラー表示Alert

- **表示条件**: error !== null
- **タイプ**: error
- **showIcon**: true

2.2.9 フッターボタン

- **キャンセルボタン**:
  - disabled: isLoading
- **ワークフロー追加ボタン**:
  - type: primary
  - disabled: !selectedFile || isLoading
  - loading: isLoading

- ラベル: "追加中..." または "ワークフロー追加"

---

3. 機能仕様

3.1 Props定義

```typescript
interface WorkflowSelectorDialogProps {
  open: boolean;
  onClose: () => void;
  onSelect: (file: FileInfo) => Promise<void>;
  availableFiles?: FileInfo[];
}

interface FileInfo {
  key: string;
  name?: string;
  size?: number;
}
```

Props説明:

Prop名	型	必須	説明
open	boolean	✅	モーダル表示状態
onClose	() => void	✅	モーダルを閉じるコールバック
onSelect	(file) => Promise	✅	ファイル選択確定コールバック
availableFiles	FileInfo[]	-	選択可能なファイル一覧（デフォルト: []）

3.2 State定義

```typescript
const [searchQuery, setSearchQuery] = useState('');
const [selectedFile, setSelectedFile] = useState<FileInfo | null>(null);
const [isLoading, setIsLoading] = useState(false);
const [error, setError] = useState<string | null>(null);
const [expandedKeys, setExpandedKeys] = useState<React.Key[]>(['/']);
const [selectedFolder, setSelectedFolder] = useState<string | null>(null);
```

State説明:

State名	型	初期値	説明
searchQuery	string	''	検索クエリ
selectedFile	FileInfo | null	null	選択中のファイル
isLoading	boolean	false	ワークフロー追加中フラグ
error	string | null	null	エラーメッセージ
expandedKeys	React.Key[]	['/']	ツリーの展開状態
selectedFolder	string | null	null	選択中のフォルダパス

3.3 計算値（useMemo）

3.3.1 folderTree

```
const folderTree = useMemo(() => {
  return buildFolderTree(availableFiles);
}, [availableFiles]);
```

説明: availableFilesからフォルダ階層構造を構築

3.3.2 filteredFiles

```
const filteredFiles = useMemo(() => {
  let files = availableFiles;

  // フォルダ選択によるフィルタリング
  if (selectedFolder) {
    files = files.filter(file => {
      const folderPath = file.key.substring(0, file.key.lastIndexOf('/'));
      return folderPath === selectedFolder;
    });
  }

  // 検索クエリによるフィルタリング
  if (searchQuery) {
    const query = searchQuery.toLowerCase();
    files = files.filter(file => {
      const fileName = file.name?.toLowerCase() || '';
      const fileKey = file.key?.toLowerCase() || '';
      return fileName.includes(query) || fileKey.includes(query);
    });
  }
```

```
  return files;
}, [availableFiles, searchQuery, selectedFolder]);
```

説明:

- フォルダ選択によるフィルタリング
- 検索クエリによるフィルタリング
- 両方のフィルタを組み合わせ可能

3.3.3 treeData

```
const treeData = useMemo(() => {
  return convertToTreeData(folderTree);
}, [folderTree]);
```

説明: FolderNodeをAnt Design Tree用のDataNodeに変換

3.4 ダイアログ初期化

3.4.1 処理フロー

```
useEffect(() => {
  if (open) {
    setSearchQuery('');
    setSelectedFile(null);
    setSelectedFolder(null);
    setError(null);
    setExpandedKeys(['/']);
  }
}, [open]);
```

3.4.2 UI動作

- ダイアログを開く度に状態をリセット
- ルートフォルダを自動展開

3.5 フォルダ選択

3.5.1 処理フロー

```
const handleSelectFolder = (selectedKeys: React.Key[]) => {
  const folderPath = selectedKeys[0] as string;
  setSelectedFolder(folderPath === selectedFolder ? null : folderPath);
  setSearchQuery(''); // フォルダ選択時は検索クエリをクリア
};
```

3.5.2 UI動作

- ユーザー操作: フォルダツリーのフォルダをクリック
- 処理:
  1. 同じフォルダを再クリック → 選択解除
  2. 異なるフォルダをクリック → 選択
  3. searchQueryをクリア
- 結果: 右ペインに選択フォルダ内のファイルのみ表示

3.6 ファイル検索

3.6.1 処理フロー

```
onChange={(e) => {
  setSearchQuery(e.target.value);
  setSelectedFolder(null); // 検索時はフォルダ選択をクリア
}}
```

3.6.2 UI動作

- ユーザー操作: 検索バーに入力
- 処理:
  1. searchQueryを更新
  2. selectedFolderをクリア
- 結果: ファイル名またはパスに検索クエリを含むファイルのみ表示

3.7 ファイル選択

3.7.1 処理フロー

```
const handleFileSelect = (file: FileInfo) => {
  setSelectedFile(file);
};
```

3.7.2 UI動作

- ユーザー操作: ファイル項目をクリック
- 結果:
  1. 選択ファイルが青色ハイライト
  2. 選択ファイル情報Alertが表示
  3. ワークフロー追加ボタンが有効化

3.8 ワークフロー追加

3.8.1 処理フロー

```
const handleConfirm = async () => {
  if (!selectedFile) {
    setError('ファイルを選択してください');
    return;
  }

  setIsLoading(true);
  setError(null);

  try {
    await onSelect(selectedFile);
    onClose();
  } catch (err) {
    console.error('ワークフロー選択エラー:', err);
    setError((err as Error).message || 'ワークフロー追加に失敗しました');
  } finally {
    setIsLoading(false);
```

```
  }
};
```

3.8.2 UI動作

- ユーザー操作: "ワークフロー追加"ボタンをクリック
- バリデーション: selectedFileがnull → エラー設定、処理中止
- 実行: 親コンポーネントのonSelect実行
- 成功: ダイアログを閉じる
- 失敗: エラーメッセージ表示

---

4. ユーティリティ関数

4.1 buildFolderTree

```typescript
const buildFolderTree = (files: FileInfo[]): FolderNode[] => {
  // ファイル一覧からフォルダツリーを構築
  // 空フォルダを削除し、名前順でソート
  // 各フォルダにファイル数を集計
};

interface FolderNode {
  name: string;
  path: string;
  children: FolderNode[];
  fileCount: number;
}
```

機能:

- ファイルパスからフォルダ階層を抽出
- ルートノードを作成
- 各フォルダのfileCountを集計
- 空フォルダを削除
- 名前順でソート

4.2 convertToTreeData

```
const convertToTreeData = (nodes: FolderNode[]): DataNode[] => {
  return nodes.map((node) => ({
    title: (
      <Space>
        {node.name}
        {node.fileCount > 0 && <Tag color="blue">{node.fileCount}件</Tag>}
      </Space>
    ),
    key: node.path,
    children: node.children.length > 0 ? convertToTreeData(node.children) : undefined,
  }));
};
```

機能:

- FolderNodeをAnt Design Tree用のDataNodeに変換
- フォルダ名とファイル数Tagを表示
- 再帰的に子ノードを変換

4.3 formatFileSize

```
const formatFileSize = (bytes?: number): string => {
  if (!bytes) return '-';
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(1)} KB`;
  return `${(bytes / (1024 * 1024)).toFixed(1)} MB`;
};
```

機能:

- バイト数を読みやすい単位（B、KB、MB）に変換

---

5. スタイル仕様

5.1 モーダル設定

```
<Modal
  width={900}
  style={{ top: 40 }}
>
```

説明:

- 幅: 900px
- 上マージン: 40px

5.2 2ペインレイアウト

```
<Flex gap="middle" style={{ height: 400 }}>
  {/* 左側: 40%、右ボーダー、paddingRight: 12px */}
  <div style={{ width: '40%', borderRight: '1px solid #f0f0f0', paddingRight: 12, overflowY: 'auto'
}}>

  {/* 右側: 60% */}
  <div style={{ width: '60%', overflowY: 'auto' }}>
</Flex>
```

5.3 選択ファイルハイライト

```
style={{
  cursor: 'pointer',
  backgroundColor: selectedFile?.key === file.key ? '#e6f7ff' : undefined,
  border: selectedFile?.key === file.key ? '2px solid #1890ff' : undefined,
}}
```

---

6. エラーハンドリング

6.1 エラーケース

エラーケース	対処方法

ファイル未選択で確定	error設定、処理中止
onSelect実行時エラー	catch、errorメッセージ表示、isLoading解除
availableFilesが空	空状態表示

6.2 バリデーション

- 確定時: selectedFileがnullの場合エラー

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/editorv2/components/WorkflowSelectorDialog.tsx`

7.2 依存コンポーネント

- Ant Design:
  - Modal
  - Input
  - Button
  - List
  - Tree
  - Alert
  - Typography
  - Tag
  - Spin
  - Space
  - Flex
- Ant Design Icons:
  - FileOutlined
  - SearchOutlined
  - FolderOutlined

- FolderOpenOutlined

7.3 親コンポーネント

- PetriNetEditorInner

---

8. 使用例

8.1 基本的な使用

```
<WorkflowSelectorDialog
  open={showWorkflowSelector}
  onClose={() => setShowWorkflowSelector(false)}
  onSelect={handleWorkflowSelect}
  availableFiles={availableFiles}
/>
```

8.2 親コンポーネントでの実装例

```
// ワークフロー選択
const handleWorkflowSelect = async (file: FileInfo) => {
  try {
    // S3からMAIML XMLファイルをダウンロード
    const maimlXml = await downloadFile(file.key);

    // XMLをパース
    const parsedMaiml = await parseMAIML(maimlXml);

    // クリックした位置にワークフローを追加
    const newNodes = convertToNodes(parsedMaiml, clickPosition);
    setNodes([...nodes, ...newNodes]);

    message.success('ワークフローを追加しました');
  } catch (error) {
    message.error('ワークフローの追加に失敗しました');
    throw error;
  }
};
```

```
// 利用可能なファイル一覧を取得
const loadAvailableFiles = async () => {
  try {
    const files = await fetchAvailableFiles(organizationId);
    setAvailableFiles(files);
  } catch (error) {
    message.error('ファイル一覧の取得に失敗しました');
  }
};
```

---

9. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで仕様書を新規作成	Cline
2026-01-29	TypeScript移植版（v1.1.0）	Development Team